Computer Security

Riassunto completo del corso

Politecnico di Milano

A.A. 2025-2026, II Semestre

Contents

1 Prefazione **2**
- 1.1 Disclaimer . . . 2
- 1.2 Keywords . . . 2

2 Introduction **2**
- 2.1 Vulnerabilities and Exploits . . . 2
  - 2.1.1 Example: exploiting a vulnerability . . . 3
  - 2.1.2 Example: A strawman software vulnerability . . . 3
- 2.2 Assets, Threats, Attacks and Threat Agents . . . 4
  - 2.2.1 What is Protected and what is Secured . . . 4
  - 2.2.2 Assets . . . 4
  - 2.2.3 Threats and Attacks . . . 4
- 2.3 Risks . . . 4
- 2.4 Security vs Cost Balance . . . 5
- 2.5 More money is not more security . . . 5
- 2.6 Trust . . . 5

3 Criptography **6**
- 3.1 Perfect Ciphers and Shannon's Theorem . . . 7
- 3.2 CSPRNG . . . 8
  - 3.2.1 PRF . . . 9
  - 3.2.2 PRP . . . 9
- 3.3 Repetitions and CTR . . . 9

4 Authentication **10**
- 4.1 To-Know Factor . . . 11
  - 4.1.1 Secure Exchange . . . 11
  - 4.1.2 Secure Storage . . . 12
- 4.2 To-Have Factor . . . 12
- 4.3 To-Be Factor . . . 13

1 Prefazione

1.1 Disclaimer

I contenuti di questo PDF sono stati compilati con il solo scopo di avere una risorsa esaustiva e facilmente consultabile degli argomenti trattati nel corso di Computer Security sostenuto dal professore Zanero: non sono in alcun modo materiale sostitutivo alle lezioni, che frequentate in presenza mi hanno permesso di capire a fondo la materia. Sconsiglio di usare esclusivamente suddetto documento per cercare di prepararsi all'esame, supporto invece l'uso di questa dispensa per fare il punto della situazione durante il periodo di studio.

1.2 Keywords

2 Introduction

How can we define what is a secure system? Usually, it follows the **CIA** paradigm to correctly define what a secure system is, the problem lies inside its definition:

- **Confidentiality** - the system should not leak information to unauthorized parties.
- **Integrity** - the system should not allow unauthorized parties to modify data **and** should not allow authorized parties to modify data in an unauthorized way.
- **Availability** - information should be always available to authorized parties.

The problem lies inside **Integrity**, if deciding whether a party is authorized or not is simple since it's a binary decision, the decision of whether a modification is authorized or not is much more complex and involves a lot of factors. Also, **Availability** is what pays off in the end. What can we consider **secure**? Something is secured if it takes an **effort** to break it: the more effort it takes, the more secure it is.

Secure System

A **Secure System** is a system that requires a lot of effort to be broken.

2.1 Vulnerabilities and Exploits

We firslty need some definitions useful to understand the world of security:

- **Vulnerability** - It is a shortcut that allows an attacker to break the system with less of the intended effort: it is something that allows to violate one of the CIA properties.
- **Exploit** - It is a specific way to use one or more vulnerabilities to accomplish a specific objective that violates one of the CIA properties.

Understanding what is and what isn't a vulnerability

Not all vulnerabilities lookalikes are actual vulnerabilities or mistakes to exploit: sometimes, something is vulnerable but the accomplishment of the attack doesn't result in a violation of the CIA properties.

2.1.1 Example: exploiting a vulnerability

When exploiting a vulnerability, we firstly need to find the security assumption that lays behind the system: in this example, the thing that ensures that the lock works is the fact that the pins are being pushed down always by the strings of the lock, so the only way to open the lock is to push up all the pins at the same time. Lockpicking is using a vulnerability of the lock to open it: this is using an **exploit**, using something to take advantage of a vulnerability to break the system.

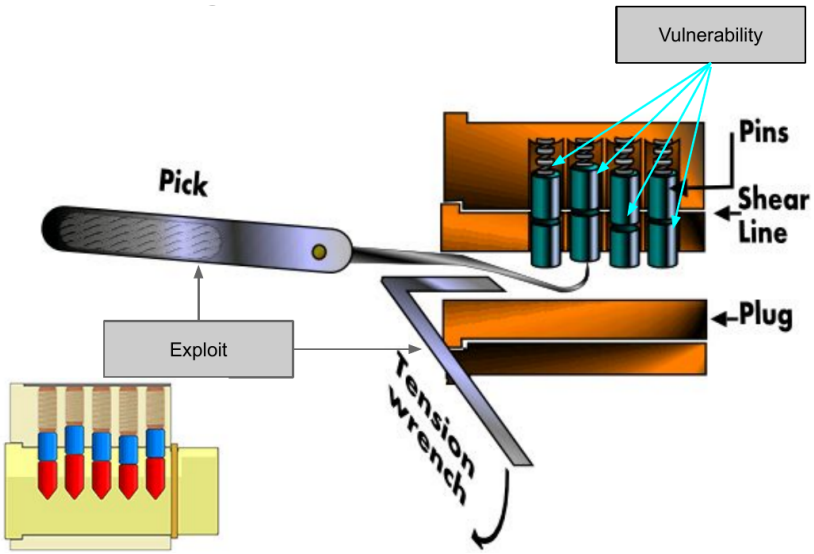


Figure 1: Lockpicking example

How can we resolve this vulnerability? Short answer: we make it harder to exploit it. Oiled locks are harder to exploit because of the deletion of friction, but still are vulnerable: some vulnerabilities are implicit in the design of the system itself.

2.1.2 Example: A strawman software vulnerability

Code

Strawman software vulnerability

```
int i;
unsigned short s;
i = atoi(argv[1]); // parse command line parameter as int
if (i == 0) { // check
    printf("Invalid number: value must be > 0\n");
    return -1;
}
s = i;
if (s == 0) { //security check
    printf("Access GRANTED!\n");
}
```

Implicit casting from int to unsigned short can cause a vulnerability (in this case, to have 0 in the variable **s** even if **i = 65536**).

2.2 Assets, Threats, Attacks and Threat Agents

2.2.1 What is Protected and what is Secured

It is crucially important that it is different to say that something is **safe** and that something is **protected**: something can be more protected but less safe, because of the presence of specific **threats** or because of its value for the individual threat (a soldier is less protected but safer against a terrorist than the polititian inside the ACV). The Cheyenne Mountain is protected against nuclear attacks, but it is not safe against rattlesnakes.

Safety agains Threats

A system is protected (and safe) always against a specific threat, it is of no sense to talk about the security of a system without considering the threat we are trying to protect against.

Watch Out!

It is total bullshit to talk about protecting a system and potential vulnerabilities without having a clear understanding of the threat models: professionals in the field of security only talk about them, not about vulnerabilities.

2.2.2 Assets

Asset

An **asset** identifies what is valuable for the organization.

We will be focusing mostly on **IT Assets**, like data, hardware and software: identifying assets is key into value them from the point of view of an attacker. Different attackers may have a different value for our different assets, it is our job to understand what could become the value of our resources and the potential loss of our assets.

2.2.3 Threats and Attacks

Threat

A **threat** is a potential violation of a CIA rule and the circumstances that can lead to said violation

A threat is usually caused by an intentional attacks, caused by attack agents (we will not cover accidents, non intentional attacks).

Hakers are not Attackers

A Hacker is someone with an advanced understanding of computers and computer networks, malicious hackers are called **black hats** (from western movies).

2.3 Risks

We just understood that **no system is invulnerable**: there will always be a vulnerability that could become a threat to our system from a specific point of view of an attacker. The only thing we can talk about is **how exposed to damage** is our system: we are talking about **risks**.

Risk

A **risk** is a statistical and economical evaluation of the exposure to damage because of the presence of vulnerabilities and threats.

Risk can be defined as a mathematical formula:

$$\text{Risk} = \text{Asset} \times \text{Vulnerabilities} \times \text{Threats} \tag{1}$$

A big problem is that the Risk term is almost always impossible to compute: we hardly know the value of our assets from an attacker point of view, we have little information on how vulnerable our system is and mostly important we are never able to identify the threats our system will encounter. It is important to note though that Assets and Vulnerabilities are **controllable variables**, the values we can actually manipulate, whereas Threats are **indepentent variables**, completely separated from our knowledge. It becomes clearer and clearer that we need to take decisions: we need to balance the urge of damage containment and the reduction of vulnerabilities to the costs they bring. This is **Security**.

2.4 Security vs Cost Balance

In security we can distinct two different costs:

- **Direct Costs** - Management, operational and equipment
- **Indirect Costs (more important)** - Less usability, slower performance, less privacy, reduced productivity

(remember when no one locked their phones when biometrics was not a thing? it was more the cost of usability than the perceived threat of having the phone opened by someone else!)

The Privacy Tradeoff Problem

When the diminuition of privacy is traded with "more security", since security is composed by privacy, it means something bad is going to happen.

2.5 More money is not more security

Sometimes a more sophisticated approach in trying to solve menaces brings only indirect costs: in an airplane, liquids are not allowed over a 100ml capacity but batteries are allowed in almost every size or form, why? Because a different approach limiting batteries would make all people avoid taking the airplane!
Lowering the security measures of a system usually doesn't happen because someone has to decide to lower it: the blame of the system having a flaw will eventually be his/hers.

2.6 Trust

It's hard to give a general definition of trust, in general or by referring specifically to computer science. In Computer Science, trust in software means that the code i wrote is "secure" (the logic itself is exactly the logic i had in mind) and also that the **compiled code** does exactly what i want to do: by securing the compiler layer of a piece of software that I wrote, I'm left with other variables that can be deemed to be external (the OS where it gets executed, the hardware that meets the piece of code that i wrote). Because of this complexity in defining what can be secured and what is to be trusted, we make **assumptions**.

Trust Issues

In general security, in any case there will be a moment where my assumptions cannot be validated further: this defines the **trust boundary** of my system and assumptions.

The trust boundary depends on our maximum effort to verify if something is trustworthy or not (if we are a small business we will have a smaller trust boundary for an agent than the penthagon).

3 Criptography

Criptography overlaps constantly with cybesecurity: they share a lot of common fields of study and the same keen objective of making secrets.

Cryptography

Cryptography is, from greece kryptos + graphein, the **art of secret writing**: it has been used mostly for commercial and military interests.

Examples of criptated messages exist from the moment writing was invented (writing itself was a form of secrecy): there existed several physical ciphers in past times that had lots of vulnerabilites. They could be **stolen**, the secret was **shared** (so if i stole this secret his friend would not know that i am **faking** to be his friend) and once an enemy figured out said secret he can **read past messages**. The best solution would be to **change the secret**, but from now on the enemy knows how the secret is formed and will try to **brute force it**. In the real world we must assume that all threat agents know the algorithm used to encrypt the messages: the only thing that is left unknown is the key. Kerckhoffs, a matematitian, stated the principles for a good cipher, some of them are:

- It must be practically (if not mathematically) unbreakable
- The key must be communicable without written notes and changeable whenever the correspondant want
- **It should be possible to make it public, even to the enemy**

The latter is justified by how in modern times information travels way faster than we can imagine and by the fact that major algorithms are known: ultimately we can arrive at the same conclusion.

Kerckhoffs' Principle

The security of a cryptosystem relies only on the secrecy of the key and **never** on the secrecy of the algorithm

We will see how, even by knowing what's going on under the hood of a system, sometimes we will never be able to decrypt a message.

Data

Data is defined in cryptography as the union of:

- **Plaintext space** - the set of possible messages, usually $\{0,1\}^l$ where l is the length of the message
- **Ciphertext space** - the set of possible ciphertext, usually $\{0,1\}^{l'}$ where usually $l' \geq l$
- **Key space** - the set of possible keys, usually $\{0,1\}^\lambda$ where λ is the length of the key

Functions

The functions used in cryptography are:

- **Encryption Function** - P x K $\rightarrow$ C
- **Decryption Function** - C x K $\rightarrow$ P

There are several types of attacks made to break a cryptosystem:

- **Ciphertext only attacks** - when the analyst has only ciphertext
- **Known plaintext attack** - when the analyst has a set of pairs plaintext-ciphertext
- **Chosen plaintext attack** - analyst can choose plaintexts and obtain their respective ciphertexts

Remember: those attacks are used by criptographers to check whether a cipher is secure or not, not by cybersecurity analysts.

3.1 Perfect Ciphers and Shannon's Theorem

Another mathematitian that contributed to the world of cryptography is **Shannon**, that demonstrates how unbreakable ciphers exist: they are called **perfect ciphers** and rely on the uniqueness of the key for each transaction.

Perfect Cipher

If we define $P(M = m)$ as the probability of observing a message m by an attacker and $P(M = m|C = c)$ as the probability that once observed a ciphertext c the message was actually m, we can define a **perfect cypher** as one that has:

$$P(M = m|C = c) = P(M = m) \tag{2}$$

This actually means that an external observer that watches m learns **nothing**: the bad news is that this definition is **non-constructive** (we don't know what a perfect cipher looks like), but we can imagine that to give away zero informations about our message we need a new key **every time a new message is sent**. Shannon theorized how a perfect cipher looks like and put it into a theorem.

Shannon's Theorem

Any symmetric encryption is perfectly secure if

- every key is used with probability $\frac{1}{|k|}$
- a unique key always maps a plaintext with a unique ciphertext

An example of a perfect cipher is the **XOR** cipher: given a message m and a key k where $|m| = |k|$, we can cipher the message by doing m XOR $k = c$. If the listener wants to read back the message he can use the same k by doing $m = c$ XOR k. This method works and by randomly using a different k every time it is a perfect cipher, however the problem now lies in **how do we send the key**: this is a crucial problem in security and **symmetric encryption** and makes this type of ciphers **hardly usable**. But we can draw some practical conclusions to this theorem: we need to find a way to **encript a message with a function using this key**, the function needs to be so hard to reverse it that it would take lots of effort (possibly more than $O(poly(\lambda))$). Finally, what we need to do as cryptographers of today is:

- Define the ideal attacker behaviour
- Assume a hard given computational problem (discrete logs, nonlinear boolean equations, large integers factoring, etc)
- Prove that any non ideal attacker solves said hard problem

Even with a perfect cipher, the system is breakable: a system is deemed **broken** if an attacker can guess the message without solving the hard problem and with a probability that is better than **random**.

3.2 CSPRNG

In reality, it is good practice to exchange a **single key** that is **kept** and **used like a seed** to generate **new keys**: the concept of generating new keys involves functions and permutations that need **randomness**. It's difficult to generate randomness in the real world of determinism, so we will need a complex algorithm that ensure enough randomeness that can be called **pseudorandom**.

CSPRNG

A **Cryptographically Safe Pseudorandom Number Generator** is a deterministic function that starts from our key $\{0,1\}^\lambda$ and generates something longer $\{0,1\}^{\lambda+l}$, with l being the **stretch**, and that cannot be used to guess the key again in at least $O(poly(\lambda))$ time.

Testing the Efficiency of a CSPRNG

We cannot prove that a function is solvable over $\{0,1\}^{\lambda+l}$ time because else we could prove that $P \neq NP$! In order to verify the security and the randomness of a CSPRNG we will need to **test it** with time and trials.

In reality we only have **candidate CSPRNGs**, we have no proof that this function exists since we cannot prove that the function itself is solvable above $\{0,1\}^{\lambda+l}$ time: we use structures and libraries that historically tend to create good algorithms and pesudorandom numbers, one building block that is used are called **PRPs** (Pseudo Random Permutations) and themselves are generated by **PRFs** (Pseusorandom Functions).

3.2.1 PRF

Considering the set of all functions that map a starting sequence $\{0,1\}^{in}$ to a finite-length encription $\{0,1\}^{out}$ and calling it F, we can say that the actual **cardinality** of this set is calculated as $|F| = (2^{out})^{2^{in}}$.
If we have for example $F = \{f : \{0,1\}^2 \to \{0,1\}^1\}$ we will have $(2^1)^{2^2} = 16$ possible entry tables to map every input string to a possible output string: since this cardinality **explodes** for big enough *in* and *out*, we can use a mechanism to generate a pseudorandom function to select one of those functions so that it's going to be hard to correctly guess what is the actual function between all the possible ones.

Pseudorandom Functions

A Pseudorandom Function is $f : \{0,1\}^{in} \to \{0,1\}^{out}$ that uses as input the input string and a λ bit seed and gives as output a sequence of characters.

3.2.2 PRP

The idea is the same as a random function, but the length of the input is the same as the length of the output and the action of permutation depends totally on the key that is given to the PRP.

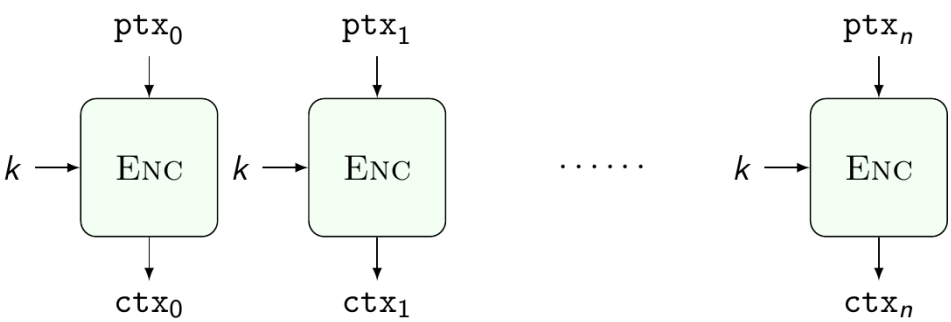
Pseudorandom Permutations

A Pseudorandom Permutation is a $p : \{0,1\}^{len} \to \{0,1\}^{len}$ and can be seen as a bijective function that uses as input the input string and a λ bit seed and gives as output a permutation of the input string.

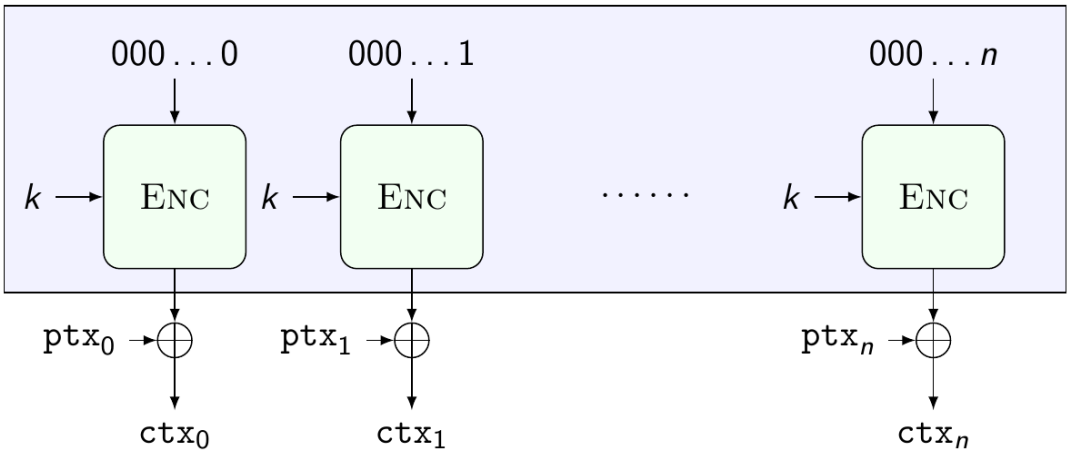
Real world PRPs are made by taking small bijective functions f and by repeating the step of inserting the input and key with its output: by computing f several times (until we are satisfied) we can obtain an arbitrarily complex function. In real world since we cannot prove that it takes $O(poly(\lambda))$ time to break the cipher, we **test them**. Nowadays PRPs are called **block cyphers**, the current standard block cypher was made through a competition. But when a cipher is to be considered **broken**? When it is possible to take a shortcut that is better than bruteforcing, that is finding a solution in less than 2^λ attempts: this is why λ needs to be a large number, the key **must be big enough**. In the real world, the current AES (Advanced Encryption Standard) actually uses 128, 192 and 256bits long keys and takes inputs of 256bits: the problem is that we have to figure out a way to divide the message in parts so that the key still is longer than the actual block of input.

3.3 Repetitions and CTR

Since the input message is necessarily longer than the key we need to find a way to divide the message into chunks of text: the most basic thing we can imagine is to divide the message into parts and use a CSPRNG to cipher every chunk individually.



Actually this method is pretty **dangerous**: by coding and decoding single chunks everytime using the same key k we could give hints to a possible attacker, since the ciphertext could contain repetitions and patterns. What we can do is to implement a mechanism that uses **additional** variable information to encode equal chunks in the same message differently, one solution is to develop a **counter** and a new encryption function that takes not only the input message and the key k, but also said counter.



4 Authentication

Firstly, let's define the difference between **identification** and **authentication**.

Authentication

Authentication is prooving that an entity is identifying itself correctly.

As an example, identification is inserting the username, authentication is correctly write the password: this is an example of **unidirectional** authentication, but there may exists also mutual/bidirectional authentications.

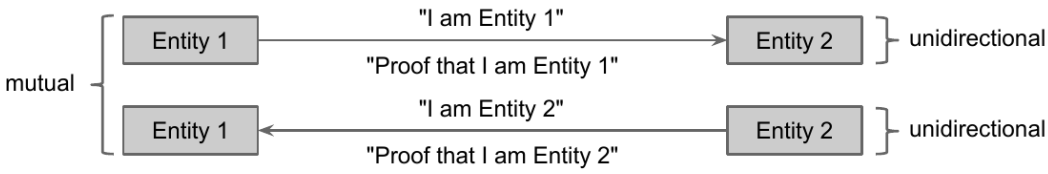


Figure 2: Mutual Authentication

The entities that enter the process of authentication may be different: i could have **human to human** auth, **human to computer** and **computer to computer**.
To authenticate we need a **proof of identity**, this may be:

- Something the entity **knows** - passowrd, PIN, handshake
- Something the entity **has** - door key, token
- Something the entity **is** - face, voice, fingerprints

4.1 To-Know Factor

Passwords and PINs are **easy to deploy and to implement** with **low implementation costs** (code is "free"): the main problem of passwords is that they are easy to be obtained by an attacker.

- **Guessing** - an attacker can guess a password by identifying similar common patterns between different passwords
- **Stealing** - a flying note on the office can be always stolen
- **Brute Forcing** - numerous attempts to decode the passwords

Also, a data breach made by someone that steals my password for a domain can be used for accessing with the same keyword to a different domain (mixture of guessing and stealing)
How can we make the password authentication process is more secure as developers and also as users?

- **Change** them continuously
- Make them **long**
- Make them **complex** with special and non-repeatable characters

Remember

Countermeasures are **costs**: humans are the weakest link in the chain of cybersecurity, they hardly remember complex passwords and to force them to write more difficult ones leads to indirect costs.

A standard problem in cybersecurity is creating a secure communication channel (such as HTTPS), we will now see how this is possible.

4.1.1 Secure Exchange

The game of authentication consists in finding a way to securely share a secret and in the **challenge-response scheme** is made possible thanks to hashing sent values.

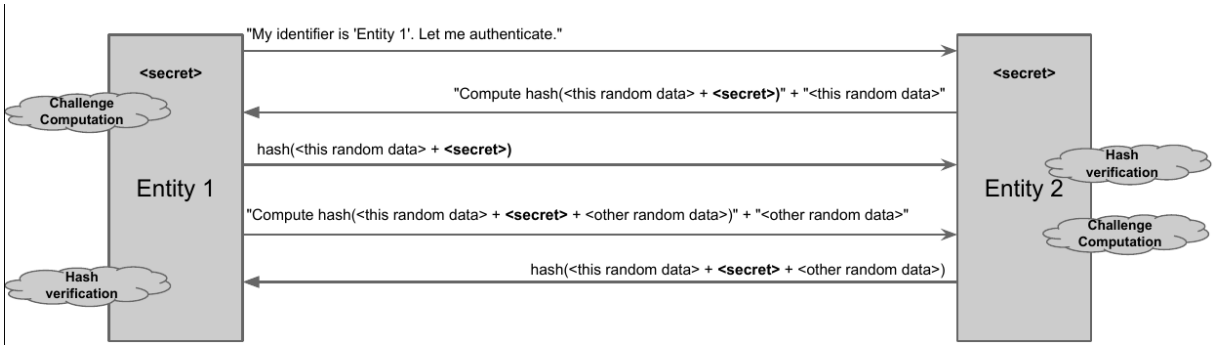


Figure 3: Challenge-Response scheme

The only problem of this system is that it is enough to breach inside Entity2 to know every secret of every user interfacing with it, meaning that this is not a feasible solution to implement in reality (see haveibeenpwned).

4.1.2 Secure Storage

I need to find a way to "remove" the option of stealing the information, i need to store my password in a **hashed way**: when i login, my password gets hashed, checked (the hashed one, not the true one) and then login happens, only the hashed version of my password gets saved. If an attacker gets to the storage of my machine he will only see my hashed version of the password, that is **unusable**.
Now the new problem is **brute force attacks**, that could try and find all passwords that match the hashed result. A solution to those brute force attacks is **salting**: adding a random number stored somewhere else in the system to the password and then hash it together.

4.2 To-Have Factor

The possession of a key is easy, cheap and comes with a good level of security, but carries a lot of problems: it's **hard to deploy** and most importantly **hard to share** at the start of the "conversation". There are many classic tecnologies that try to cope with those problems:

- **One-Time Password Generators** - user has a device with a counter that is synchronized with the one of the authenticator and changes every 30-60 seconds, if communicated counter + key is correct then user is authenticated

  Synchronization

  If the device is not kept synchronized overtime then it breaks

- **Smart Cards** - the key lies inside a physical integrated circuit and the authentication is done by a challenge-response protocol

  Tampering

  If the device is not tamper proof, then it should not be trusted

- **Static OTP Lists** - static tables with codes that corresponds to codes saved both by the user and the host (outdated)

  Easy to steal

  A simple look at the physical card and all your codes are gone, a simple listen of some communications between you and the host and at least one code is gone, and so on

- **Token OTPs** - the same key logic of OTP generators, but this time using software that is embedded and closed

  Difficult to defend

  Since the OPT generator is your phone and since it is a multi-purpose device, it is harder to secure and protect (see malicious software)

There are also a lot of **scam attacks** and **impersonating thefts** linked with physical devices (see the SIM scam).

4.3 To-Be Factor

Nowadays the most common ways to authenticate are linked to identity as **something we have**, biometric authentication: this has a high level of security and requires no extra hardware to carry around (we are the key!). Another important thing is that it's **effortless**: since humans are not keen to normally authenticate during their lifespan this is a big advantage and this is the main reason that led biometrics to be the most used form of authentication used by humans. There are a lot of issues linked to this particular factor: it's **hard to deploy safely**, has an **invasive approach** thanks to its nature, it **can be cloned** and also the biometrics **change overtime**. The main characteristics that can be used in biometrics authentication are:

- **Fingerprints**
- **Face geometry**
- **Palm print** (hands are unique in human beings!)
- **Retina scan** (internal part of the eye, including blood vessels)
- **Iris scan** (colored part of the eye, unique pigmentations and defects)
- **Voice analysis** (with lots of difficulties)
- **DNA** (with the exception of identical twins)

How we are and how we act

There are charateristics that define **how we act** (like how we walk, how we write), but those characteristics are hard to measure in computer systems.

The main problems tied with this type of authentication is **portability**: not all devices use a specific scanner to analyze a certain type of human characteristic. It's also **costly** both to integrate in our daily technologies and also intrisically (retina scanners cost a lot). Also, the main disadvantage of biometrics is that it is **probabilistic and non-deterministic**: if passwords are either right or wrong biometric measurements rely on tresholds and false positives and negatives (here called **false acceptances** and **false rejections**). Obviously **false acceptances are a vulnerability** but false rejections are the true effort-generators (users with their phones do not experience false acceptances, but only false rejections!): the main objective of a programmer should be to define the right **threshold** that lowers enough the false acceptances and still doesn't piss off the user by making him unlock the smartphone 5 times (false acceptances are **high** on smartphones, but on purpose!).

Single-User Devices Threat Models

Single-user devices are meant to have cheap authentication sensors because the threat model of a smartphone is composed by thieves, family members, law enforcment: depending on our status (VIPs or not) a thief will not try to unlock the system, family members will not try to do us damage (but if they want to, they will and they would be **unstoppable**) so they do not make up to our threat model, also law enforcement are not part of the threat model analyzed (too powerful to defend against). Against thieves, this is designed correctly.

Another big flaw is that we have 10 fingers only: **cloning** is the main flaw of biometrics, once someone gets the characteristics of how we are, we get completely **violated** (the design of biometric authentication is complex: where do i store the data that defines who i am? How can

i defend this from attacks? Is it to valuable to store this? If someone steals this information what is the damage?).

A Take on Global Security

When interests are not aligned, vulnerabilities start to emerge.
Facebook uses OAuth2 as a common way to authenticate, generating tokens that **never** expire: is this a secure way to authenticat? The answer is a big **yes for Facebook**, since their business model is made to target and analyze the user interests, and a big **no for us users**, since the token once stolen can be used by someone else without us noticing. Keep in mind that in the real world **this is exactly how it should be done** and this is why Facebook had such success: because it **doesn't care about single user accounts**, but only their informations and habits. In this case, the vulnerability that is generated is on **our side**.